DealBreakers

Listo Deal Room — Source-Grounded Agentic Travel Seller

Team members: Muhammad Asad Majeed • Muhammad Maaz • Abdul Azeem Makarim • Abdussalam Popoola

---

Challenge and Objective

DealBreakers is an autonomous seller for the **Listo Deal Room** challenge. In each match an AI buyer arrives with a private persona, hidden budget ceiling, travel preferences, and willingness to walk away. The seller has at most 15 turns to discover the buyer's needs, source a real travel package from live MCP integrations, quote a marked-up offer, negotiate objections, and close.

The scoring objective is deliberately multi-objective: **Close** is worth 50 points, **margin** is worth 30 points, and **satisfaction** is worth 20 points. Margin is computed as $(q - c)/(B - c)$, where q is the quoted total, c is the verified live MCP cost, and B is the buyer's hidden budget. This means the agent cannot simply undercut to close, nor can it overprice blindly. It must infer the buyer's ceiling from conversation, preserve trust, and know when a lower-margin close is better than a high-margin walk-away.

Agentic Design Philosophy

Our core design decision was to build a **controlled agentic system**, not a single free-form chatbot. The top-level `SellerAgent` owns the negotiation state and round-by-round control flow. Around it are specialised evaluator agents that each perform one bounded reasoning task: extract the buyer profile, read the latest emotional/negotiation signal, choose among live listings, advise a markup, draft a message, and critique the draft before it is sent.

This gives us the benefits of agentic reasoning without giving the language model unchecked authority over facts or prices. The LLM can judge nuance, tone, and trade-offs, but live inventory, verified cost, source URLs, offer structure, and hard guard rails remain deterministic Python code. Every outbound offer is built from a canonical `ListingCandidate`, validated by Pydantic models, and attached to source receipts before reaching the Deal Room API.

End-to-End Round Loop

Each buyer message passes through the same pipeline:

1. **State update.** The agent stores the transcript, latest buyer message, previous seller messages, quotes, price objections, current candidate, pivots, optional car add-on, and search key.
2. **Profile extraction.** A regex baseline in `profile.py` extracts obvious facts such as destination, party size, trip style, nights, budget, must-haves, luxury signals, and price sensitivity. The `ProfileEvaluator` then performs a structured LLM extraction over the transcript and merges only typed fields into the profile.
3. **Echo guard.** If the extracted budget resembles one of our own previous quotes or discount deltas, it is discarded. This prevents the LLM from mistaking "that price is too high" or a repeated seller total for the buyer's true budget ceiling.
4. **Buyer read.** The same evaluator reads the latest message for mood, tone, price resistance, overcharge suspicion, impatience, close signal, and main objection. These values decide whether to ask another question, hold firm, concede, or pivot.
5. **Search decision.** The controller searches only when the profile has enough signal or when a fit objection means the current product is wrong. It caches a search key so the agent does not repeatedly re-source for unchanged requirements.
6. **Live MCP search.** `McpSearchEngine` ranks travel MCPs by trip type, discovers relevant tools, fills arguments only from known profile facts, and retries with broader arguments if the first pass returns nothing.
7. **Candidate normalisation.** `catalog.py` converts heterogeneous MCP responses into canonical candidates with source, product type, name, URL, total price, location, nights, board basis, star rating, review score, operator, and amenities.
8. **Shortlist selection.** A deterministic scorer ranks candidates for product type, destination, budget headroom, amenities, quality, and luxury fit. The `ShortlistEvaluator` can then choose the best item from the shortlist, with guard rails that avoid overly expensive bases when the budget is unknown.
9. **Pricing.** `PricingStrategist` combines deterministic opening anchors, quote history, buyer read, remaining rounds, known budget, and LLM pricing advice. A markup ladder enforces never quoting higher after a quote, visible concessions on real price pushback, firm holds when the buyer is close, and endgame close-first pricing.
10. **Offer construction.** The structured offer is built from verified MCP cost plus markup. If a car is required, EconomyBookings or TravelSupermarket car hire is added as a separate source-backed component.
11. **Message drafting and critique.** The `MessageComposerLLM` writes a short seller message from a precise turn intent and verified package summary. The `MessageCritic` reviews it for tone, concision, factual grounding, concession

framing, and amenity wording before send.

12. **Deal Room turn.** The final `SellerTurn` includes text plus an optional structured offer. The API returns the buyer's response, quote metadata, and result state; the agent logs the turn and repeats until accept, walk, or round limit.

Specialised Agents and Responsibilities

Agent / module	Responsibility
`SellerAgent`	Central orchestrator. Maintains state, controls round pacing, decides when to ask, search, offer, concede, pivot, add a car, stop, or log.
`ProfileEvaluator`	Structured LLM agent for buyer facts and psychology. Extracts typed profile fields and reads resistance, impatience, close signal, and objection type.
`McpSearchEngine`	Tool-using inventory agent. Ranks MCP servers/tools, fills tool arguments from the buyer profile, retries broadened searches, and finds car hire when needed.
`CandidateScorer`	Deterministic fit evaluator. Scores listings by product match, destination, budget headroom, must-have amenities, review quality, and luxury level.
`ShortlistEvaluator`	Selection agent. Chooses the most sellable candidate from scored live options while avoiding structurally unclosable high-cost bases.
`PricingStrategist`	Negotiation agent. Infers ceiling, sets markup, applies concession ladder, handles endgame pricing, and resets anchors when product class changes.
`MessageComposerLLM`	Seller voice agent. Converts a turn intent and verified facts into a concise, persuasive buyer-facing message.
`MessageCritic`	Internal review agent. Rejects invented facts, weak concession framing, banned chatbot phrasing, and hesitant amenity language before the message is sent.
`models.py`	Contract layer. Validates Deal Room payloads, structured offers, source receipts, holiday/tour/car schemas, and quote/result objects.

Live Inventory and Grounding

The agent uses real MCP data rather than invented package descriptions. The MCP layer knows the public travel servers: TravelSupermarket, trivago, Kiwi, EconomyBookings, and TourRadar. For each server it performs the JSON-RPC MCP initialise flow where required, lists tools, selects travel-relevant tools, and calls them with arguments derived from the buyer profile.

Because each MCP returns different shapes, `catalog.py` contains normalisers for structured offers, text summaries, generic nested dictionaries, and TourRadar's embedded JSON text. The output is always a `ListingCandidate`. This canonical layer is important: downstream scoring, pricing, offer construction, and message grounding all work against one internal schema regardless of source.

Only candidates with a real total price, real URL or deep link, and enough identifying data can become offers. The structured offer stores the original cost as `priceTotal`, applies markup separately as `markupPct`, and records source receipts. This separation ensures the buyer-facing negotiation can be persuasive while the competition API can still verify the true cost basis.

Negotiation Strategy

The strategy is built around the scoring reality: close first, then margin, then satisfaction. That does not mean discounting early. It means choosing offers that are closable and preserving enough markup room to make concessions feel meaningful.

- **Qualify only when useful.** In rounds 1–2 the agent may ask for missing critical facts. If the buyer is impatient or enough is known to search, it moves directly to a concrete offer.
- **Open on a sellable base.** The first quote uses a calibrated anchor. Luxury buyers and low-resistance buyers can support higher markup; price-sensitive or impatient buyers receive a softer opening.
- **Prefer cheapest qualifying quality.** A lower verified cost creates more margin headroom and more room for visible concessions. The system avoids picking the most expensive luxury item when a mid-priced option still satisfies the quality bar.
- **Use sticky candidates.** Once a product has been quoted, background searches do not randomly swap it. Unexplained hotel changes reduce trust. The candidate changes only for a fit rescue or deliberate price pivot.
- **Concede on total, not just percentage.** Buyers react to the total price they see. The markup ladder ensures price concessions are visible in pounds and never accidentally quote higher after pushback.
- **Hold when close.** If the buyer is asking detail questions or showing acceptance signals without serious price resistance, the agent holds the price instead of donating margin.

- **Pivot when markup cannot solve the problem.** If the buyer keeps objecting and markup is already low, the agent searches for a cheaper base product. A successful pivot must produce a dramatic total-price drop, not a sideways swap.
- **Endgame close-first.** In the final rounds the markup cap tightens. A closed low-margin deal beats a high-margin walk-away under the challenge scoring.

Handling Buyer Psychology

Buyer messages are not treated as plain text commands. The agent tracks resistance, impatience, fit objections, suspicion of overcharging, and close signals. These readings alter behaviour:

- Impatient buyers get offers faster and fewer qualifying questions.
- Price objections increase the consecutive objection counter and trigger meaningful concessions or pivots.
- Fit objections can unlock a re-search and candidate replacement.
- Close signals cause the pricing ladder to hold rather than concede unnecessarily.
- Overcharge accusations produce stronger total-price caps because trust is at risk.

The message layer also adapts to the buyer. The composer is instructed to sound like a sharp human travel seller rather than a generic assistant, avoid filler empathy, mirror bluntness when appropriate, stay within one to three sentences, and never mention internal markup, cost, or scoring. The critic enforces the same rules and improves weak drafts.

Factual Safety and Robustness

The system has several protections against common agent failure modes:

- **No fabricated offers.** Specific hotel, tour, car, price, and amenity claims must come from candidate summaries produced from live MCP data.
- **Typed structured outputs.** LLM extraction, buyer reads, pricing advice, shortlist verdicts, and draft reviews all use Pydantic schemas.
- **Deterministic fallbacks.** If an LLM call fails, the controller can still ask, search, price, compose, and continue using rule-based logic.
- **Schema validation.** A `StructuredOffer` must contain exactly one primary product, non-empty source receipts, positive cost, and valid holiday/tour/car fields.
- **Source receipts.** Every offer carries MCP source URLs and prices, so margin is auditable.
- **Conversation death detection.** If the buyer has clearly left even though the API continues, the agent stops burning rounds.
- **Amenity translation guard.** Canonical tags are translated into confident buyer language, e.g. `close_to_beach` is pitched as beach/on-beach language and `jacuzzi` can be framed as spa/wellness when relevant, without inventing unsupported details.

Why This Counts as Agentic

The approach is agentic because the system repeatedly observes, reasons, acts, receives feedback, and updates its plan under uncertainty. It uses tools to gather live external data, maintains memory across turns, decomposes the negotiation into specialised cognitive roles, and changes tactics based on buyer reactions. It is also deliberately constrained: the central controller decides when each agent may act, typed schemas restrict what the LLM can return, and deterministic code owns high-risk operations such as pricing guards and API payload construction.

In short, DealBreakers is a **multi-agent negotiation controller**: LLM agents provide judgement where language and psychology matter, MCP tool use supplies real inventory, and deterministic guard rails keep the conversation factual, priced correctly, and optimised for the competition score.

DealBreakers — UCL. Built for the Listo Deal Room competition. Stack: Python, Pydantic, OpenAI, live travel MCP integrations, Deal Room buyer API.